



Rapport de Projet

Développement d'une Application de Chat Sécurisée

Introduction à la Sécurité

Réalisé par :

Néha Sougoumar, Anyce Ekomono, François Godin, Valentin Guillon, Maëva Himeur, Dounia Hullot

Date de Soumission :

11 Décembre 2024

Table des matières

Introduction	3
1. Répartition des Contributions	3
2. Technologies utilisées	3
3. Dépôt GitLab	4
4. Utilisation du chat	4
Fonctionnalités Implémentées	5
5. Format des informations transmises	5
6. Fonctionnalités	5
(a) Préconnexion	5
(b) Connexion	5
(c) Déconnexion	5
(d) Envoi de Messages	6
(e) Création de Groupes	6
Protocole de Sécurité	7
7. Chiffrement	7
(a) Chiffrement de la clé de groupe	8
(b) Chiffrement des messages	8
8. Étape de communication sécurisée	8
Conclusion et Perspectives	9
9. Conclusion	9
10. Figures et Liens	9

Introduction

Introduction

Le projet a pour objectif de concevoir et développer une application de mini-chat sécurisé permettant à un ensemble d'utilisateurs de s'échanger des messages sécurisés.

Les principales fonctionnalités recherchées étaient le chiffrement de bout en bout des messages échangés, l'ajout d'une interface minimaliste avec la possibilité d'initier une conversation avec un ou plusieurs utilisateurs, la mise en place d'un serveur pour répondre aux différentes requêtes des clients, et enfin, la mise en place d'une application de client malhonnête.

Nous détaillerons dans ce rapport les fonctionnalités implémentées ainsi que les choix d'implémentation qui ont été fait.

1 Répartition des Contributions

L'équipe a collaboré sur divers aspects du projet, chacun apportant son expertise :

- **Interface Graphique** : Anyce, Dounia, Maëva.
- **Partage de clés** : Néha.
- **Implémentation principale (serveur, client, gestion des connexions)** : Valentin, François.
- **Rédaction du protocole** : L'ensemble de l'équipe.

2 Technologies utilisées

Le projet est développé en **Python**, en s'appuyant sur plusieurs modules et librairies :

Modules principaux :

- **Socket** : Établir les connexions clients / serveur et permettre les échanges d'informations via le réseau.
- **Threading** : Faire tourner plusieurs tâches en parallèle dans notre programme, notamment pour la gestion de plusieurs connexions réseaux.
- **PyNaCl** : Chiffrement symétrique des messages via SecretBox. ([voir la documentation](#))
- **json** : Structurer nos données en texte pour faciliter les échanges.

Interface graphique :

- **Tkinter** : utilisé pour développer l'interface graphique de l'application. Cette bibliothèque native de Python offre des outils puissants et flexibles pour la création d'interfaces utilisateur intuitives.

3 Dépôt GitLab

Le code source complet du projet est hébergé sur GitLab, accessible via le lien suivant : <https://code.up8.edu/fgodin/p8-mini-chat>

4 Utilisation du chat

Pour utiliser le chat sécurisé, il est nécessaire de démarrer d'abord le serveur, puis de lancer les clients. voici les étapes :

Pour démarrer le serveur, ouvrez un terminal et exécutez :

```
python3 serveur.py
```

Le serveur s'exécutera et attendra les connexions clients.

Pour démarrer le client, ouvrez un ou plusieurs terminaux et exécutez :

```
python3 client.py
```

Le client démarrera dans une interface graphique permettant de se connecter au serveur.

Fonctionnalités implémentées

5 Format des informations transmises

Les informations transitant au travers des sockets suivent un format structuré et uniforme. Chaque message est un **dictionnaire Python**, converti en chaîne de caractères avant d'être envoyé. Ce dictionnaire contient trois champs principaux :

- **L'émetteur** : identifiant ou pseudonyme représentant qui est à l'origine de la requête.
- **Le destinataire** : nom du groupe ou de l'utilisateur ciblé.
- **Le contenu du message** : texte ou données envoyées.

Pour voir le format :2

6 Fonctionnalités

L'application implémente les fonctionnalités suivantes :

(a) Préconnexion

La préconnexion se déclenche automatiquement au lancement de l'application. Le client établit une connexion avec le serveur en utilisant des sockets. Lors de cette étape, le serveur crée une instance temporaire pour chaque client connecté. Un identifiant unique, est attribué au client comme nom temporaire. Cet identifiant est ensuite transmis au client, qui l'utilise jusqu'à ce qu'un nom d'utilisateur définitif soit validé. Le processus est représenté à la Figure 3 et 4.

(b) Connexion

La connexion s'active lorsque l'utilisateur valide son pseudo via l'interface de connexion. Le client envoie une requête au serveur contenant le pseudo et une clé publique fraîchement générée. Le serveur analyse cette requête en vérifiant si un client portant le même pseudo est déjà connecté. En fonction des résultats :

- Si le pseudo est déjà utilisé et que le client est connecté, le serveur rejette la requête et retourne un message d'erreur.
- Sinon, le serveur valide la connexion, puis détermine s'il s'agit d'une première connexion ou d'une reconnexion. Dans chaque cas, les informations nécessaires sont transmises au client, permettant une gestion fluide des utilisateurs.

Le processus est représenté à la Figure 5 et 6.

(c) Déconnexion

La déconnexion se produit lorsque l'utilisateur ferme l'interface. Le client envoie un message au serveur pour indiquer son intention de se déconnecter. Le serveur met à jour son état interne pour marquer ce client comme déconnecté. Une fois cette étape terminée, le programme client peut se fermer en toute sécurité. Le processus est représenté à la Figure 7 et 8.

(d) Envoi de Messages

L'envoi de messages est déclenché lorsque l'utilisateur valide un message via l'interface de messagerie. Les messages ne peuvent être échangés qu'à l'intérieur d'un groupe spécifique. Lorsqu'un utilisateur envoie un message, le client transmet ce message au serveur, qui se charge de le distribuer à tous les membres du groupe. Le processus est représenté à la Figure 9 et 10.

(e) Création de Groupes

La création de groupes est initiée lorsque l'utilisateur valide un nom via l'interface dédiée. Le client envoie une requête au serveur contenant le nom du groupe souhaité. Le serveur vérifie alors si un groupe portant le même nom existe déjà :

- Si le nom est déjà utilisé, le serveur retourne une erreur.
- Si le nom est disponible, le serveur crée le groupe, notifie tous les clients de son existence, et envoie une confirmation au créateur. Le client créateur génère ensuite une clé pour le groupe et la stocke localement.

Le processus est représenté à la Figure 11 et 12.

Protocole de Sécurité

7 Chiffrement

Deux types de chiffrement sont utilisés dans ce projet :

- Chiffrement asymétrique RSA : Garantit un échange sécurisé de la clé d'un groupe de discussion.
- Chiffrement symétrique à l'aide de **PyNaCl** (Secret Box) : Permet le chiffrement et le déchiffrement des messages dans un groupe de discussion à l'aide d'une clé commune.

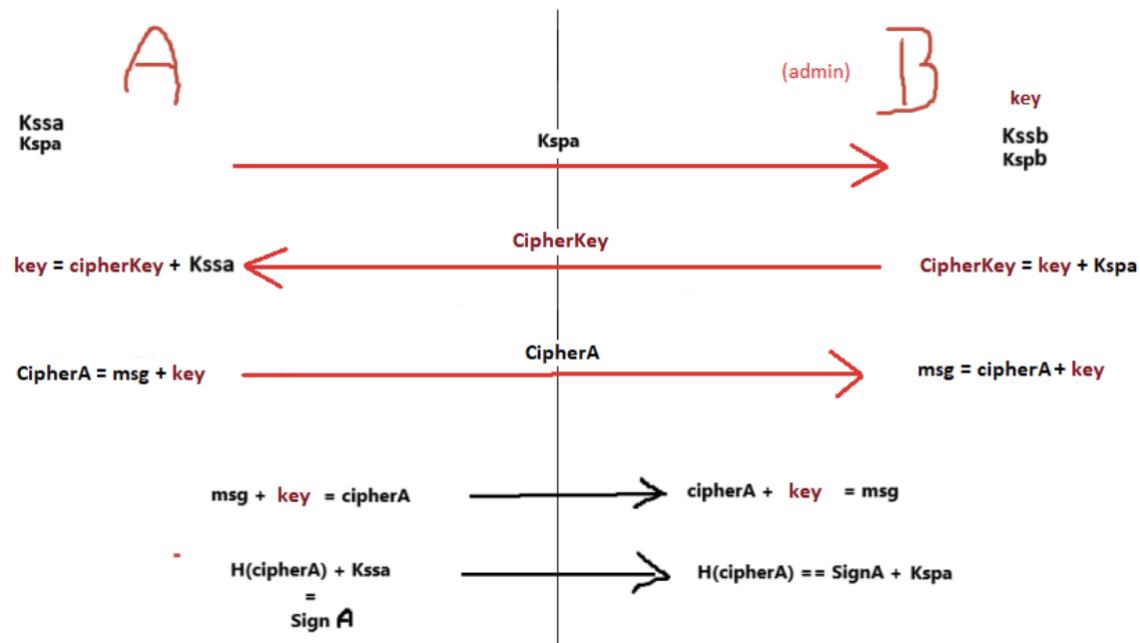


FIGURE 1 – Schéma du chiffrement entre deux clients

- Kss : Clé secrète signature
- Ksp : Clé publique signature
- $H(...)$: Fonction de hachage pour RSA
- key : Clé de chiffrement symétrique

(a) Chiffrement de la clé de groupe

Chaque client du chat possède une paire de clés RSA : une **clé publique** et une **clé privée**.

Un client transmet sa clé publique RSA lorsqu'il se connecte au serveur et ce dernier la stocke en mémoire.

Lorsqu'un client demande à rejoindre un groupe existant, notre serveur s'arrange pour transmettre la bonne clé publique à l'administrateur du groupe. Celui-ci l'utilise pour chiffrer et envoyer la clé de groupe.

Le client à l'origine de la demande, utilise sa clé secrète pour déchiffrer cette même clé du groupe.

(b) Chiffrement des messages

Pour les échanges de messages, nous avons opté pour le chiffrement symétrique via **Secret Box** de la bibliothèque **PyNaCl**.

Un nonce est utilisé pour assurer l'unicité de chaque message.

Cette librairie python nous permet de chiffrer et déchiffrer des messages à l'aide d'une clé secrète identique pour l'ensemble du groupe de discussion.

Chaque client possède donc la clé symétrique des groupes auxquels il appartient.

8 Étape de communication sécurisée

Le etapes de communication sécurisé entre les clients s'opère en plusieurs étapes :

1. **Échange de clés RSA** : Lorsqu'un utilisateur se connecte à un serveur ou rejoint le serveur, il échange sa clé publique avec le serveur qui les garde . Ces clés publiques sont utilisées pour établir des communications sécurisées.
2. **Création et échanges de clés de groupe** : Lors de la création d'un groupe, une clé de groupe symétrique est générée par 'l'admin'(le créateur du groupe). Cette clé est ensuite partagée entre tous les membres du groupe au travers d'une requête vers l'admin. Elle est utilisée pour le chiffrement des messages au sein de ce groupe. L'envoi de cette clé ce fait en utilisant le chiffrement RSA
3. **Chiffrement et envoi de messages** : Lorsqu'un utilisateur souhaite envoyer un message à un autre membre du groupe, il utilise la clé de groupe symétrique pour chiffrer le contenu du message avec la méthode *Secret Box*. Le message chiffré est ensuite transmis au serveur, qui le redirige vers les autres membres du groupe.
4. **Déchiffrement du message** : Les membres du groupe, disposant de la clé de groupe partagée, peuvent déchiffrer le message reçu et ainsi en prendre connaissance.

Conclusion et Perspectives

9 Conclusion

Résumé des réalisations

Au cours de ce projet, nous avons développé un système de messagerie sécurisé utilisant le chiffrement asymétrique (RSA) et symétrique (Secret Box de PyNaCl). Le système permet une communication sécurisée entre plusieurs utilisateurs via des sockets réseau. Chaque utilisateur dispose d'une clé publique et privée pour l'échange sécurisé de clés, tandis que les messages sont chiffrés à l'aide de la clé symétrique partagée pour chaque groupe d'utilisateurs. Le projet a permis de concevoir une interface utilisateur fonctionnelle, offrant une gestion des groupes et une messagerie sécurisée entre les membres.

Améliorations possibles

Bien que le système de base soit fonctionnel et relativement sécurisé, plusieurs améliorations peuvent être envisagées. Tout d'abord, le projet ne comporte pas actuellement de mécanisme pour tester la résistance du système à des attaques de type client malveillant. De plus, la gestion des clés de chiffrement pourrait être renforcée, notamment par l'intégration d'un système de renouvellement périodique des clés pour limiter le risque en cas de compromission.

- **serveur.py** : Ce fichier contient le code du serveur qui gère la connexion des clients, la réception et l'envoi de messages, ainsi que la gestion des groupes.
- **client.py** : Ce fichier contient le code du client, qui inclut l'interface graphique pour l'utilisateur, la gestion des connexions, de l'envoi des messages et de chiffrer les messages.
- **commonlib.py** : Ce fichier contient des fonctions utilitaires partagées entre le client et le serveur, telles que la gestion des dictionnaires de données échangées.
- **rsa.py** : Ce fichier implémente les fonctions liées à la cryptographie RSA, permettant de sécuriser les échanges de clés et les messages.
- **secret-box.py** : Ce fichier gère le chiffrement et déchiffrement des messages et des données à l'aide de techniques de cryptographie symétrique. Elle utilise la librairie PyNaCl et est utilisée pour sécuriser les messages envoyées dans les groupes.

10 Figures et Liens

Pour avoir tout les schémas ensembles réalisés sur Canva : https://www.canva.com/design/DAGXBbY1c7I/NICZWVEpzDu46XDtHZWoFQ/view?utm_content=DAGXBbY1c7I&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utm_id=hb23c7780be Pour avoir l'interface graphique conçu à l'aide figma : <https://www.figma.com/design/q2kvaDPG4YNI1hiwcr2ugg/Untitled?node-id=0-1&node-type=canvas&t=dNSDd36UtqXglvNp-0>

Exemple

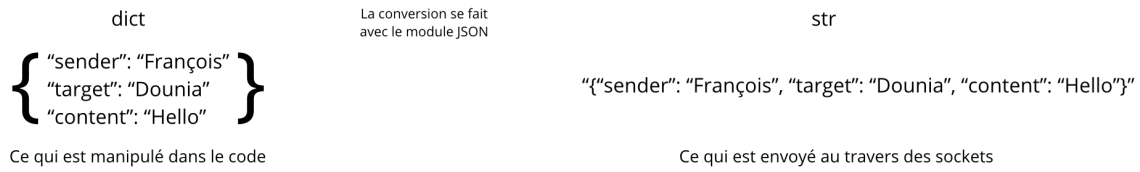


FIGURE 2 – Format des message

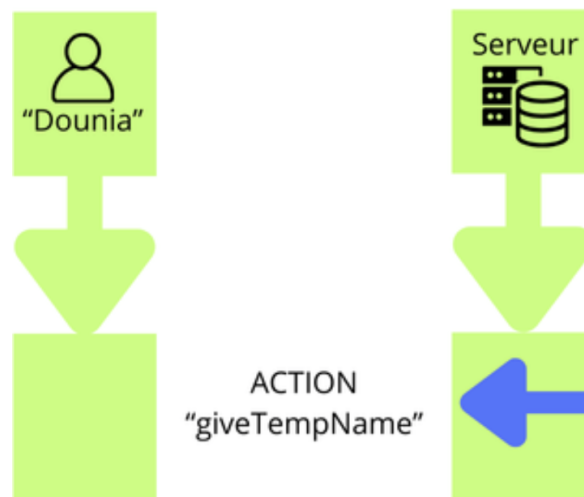


FIGURE 3 – Préconnexion au serveur

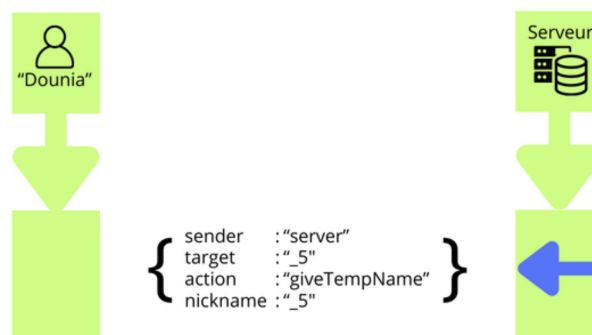


FIGURE 4 – Préconnexion au serveur (avec format)

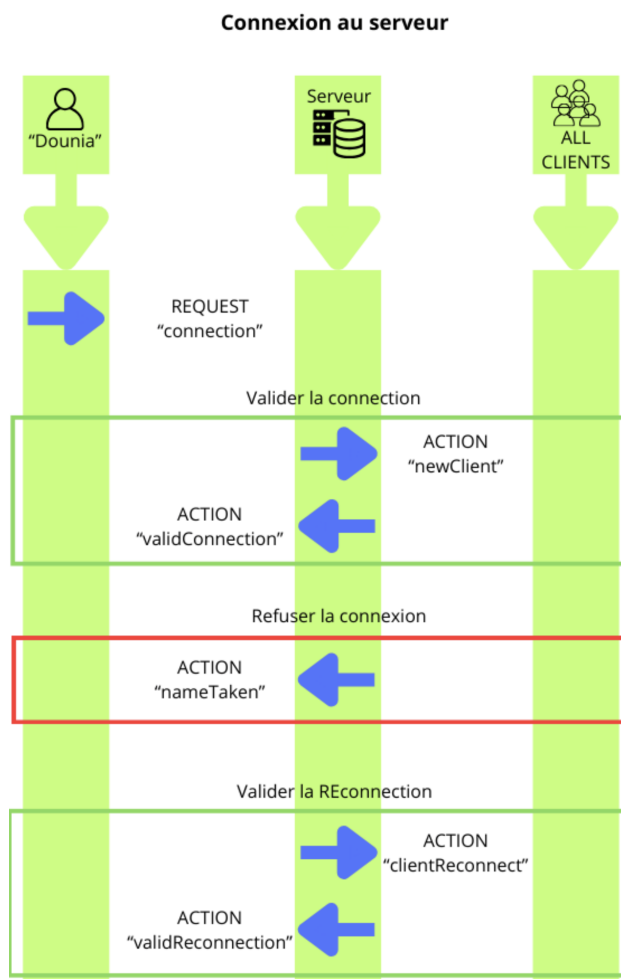


FIGURE 5 – Connexion au serveur

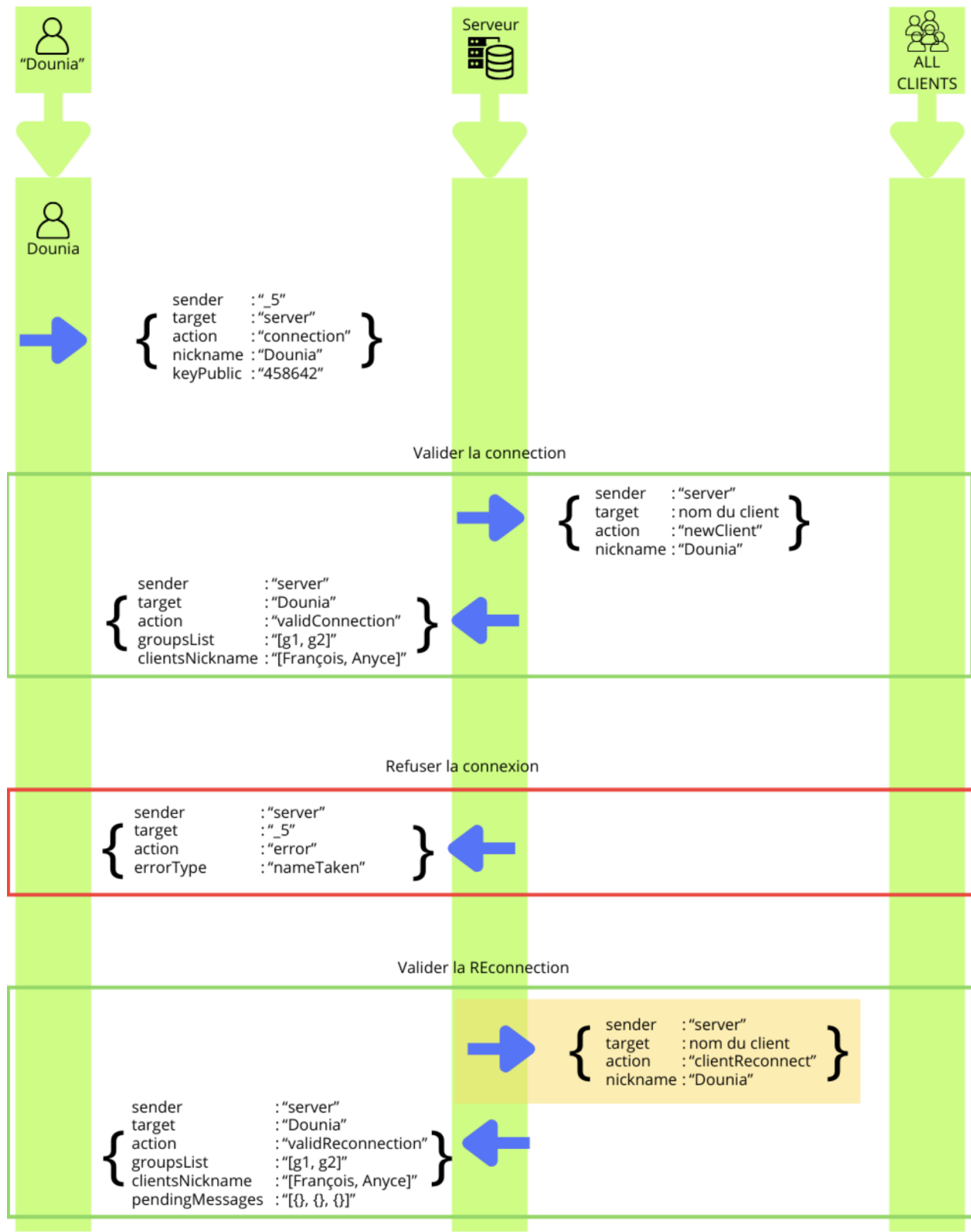


FIGURE 6 – Connexion au serveur (avec format)

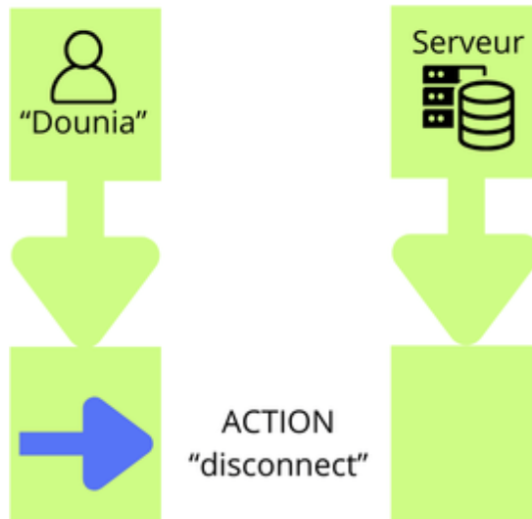


FIGURE 7 – Enter Caption

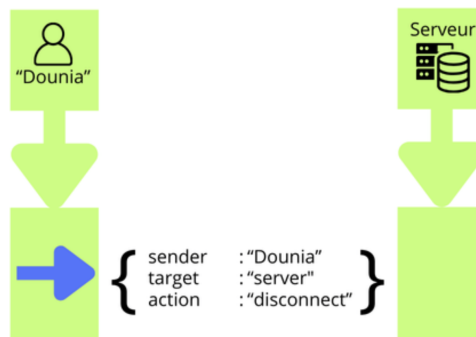


FIGURE 8 – Déconnexion (avec format)

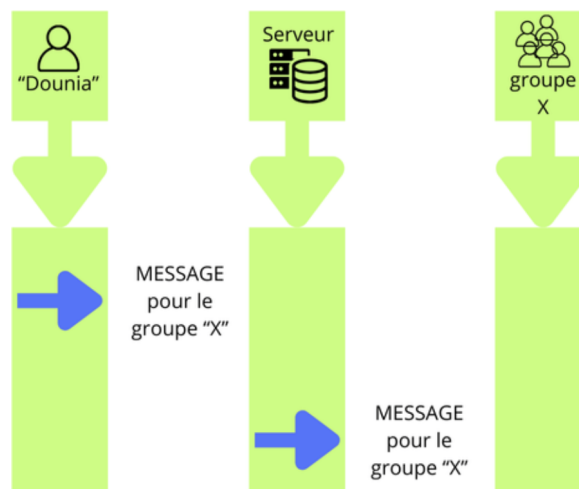


FIGURE 9 – Envoyer un message

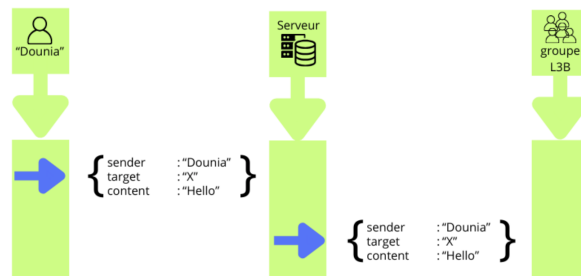


FIGURE 10 – Envoyer un message (avec format)

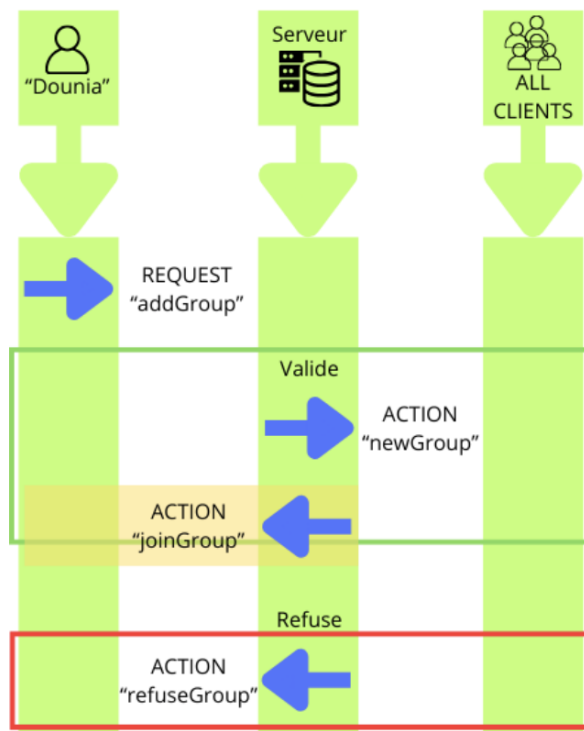


FIGURE 11 – Création de Groupe

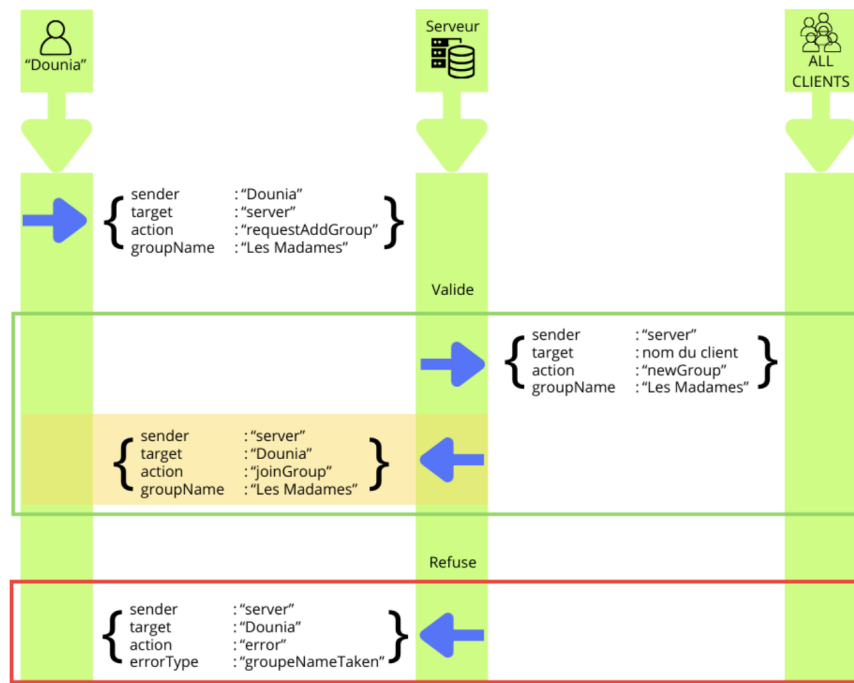


FIGURE 12 – Creation de Groupe (avec format)

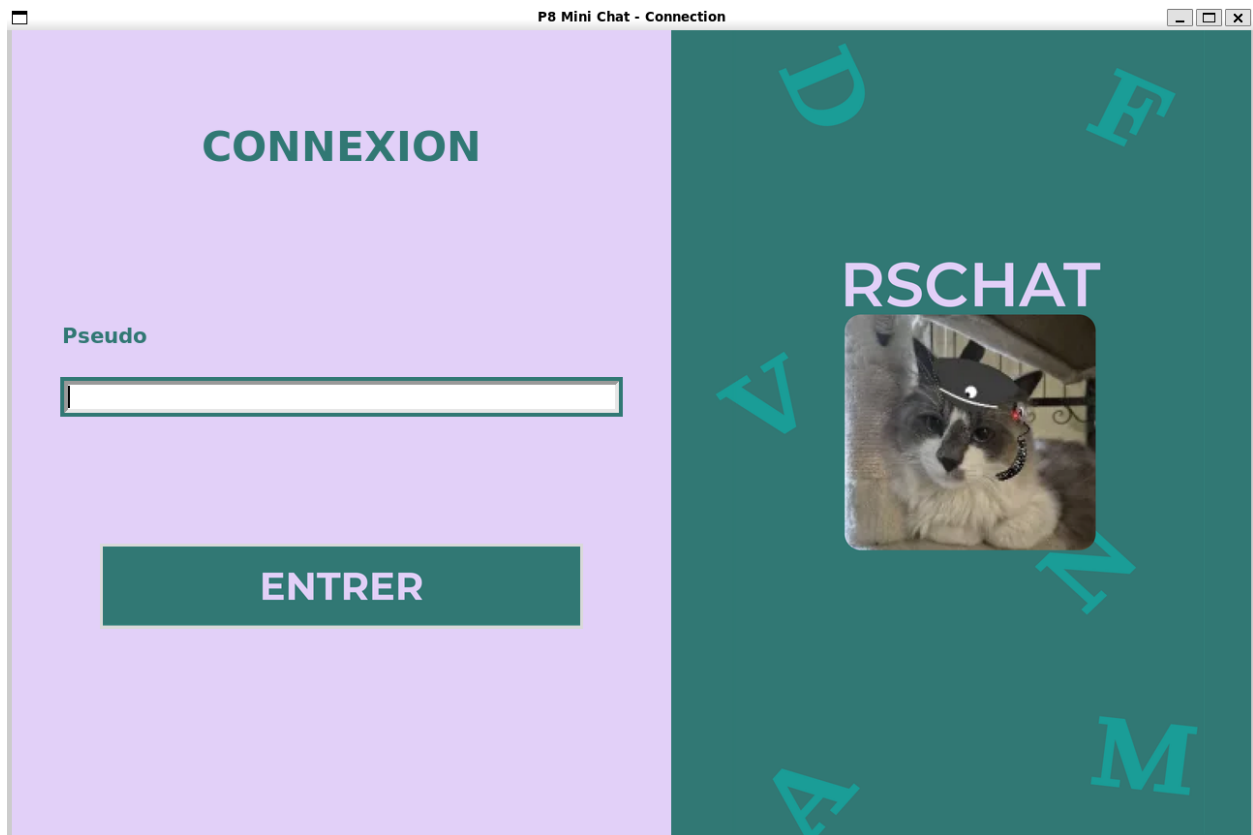


FIGURE 13 – Interface graphique de connexion

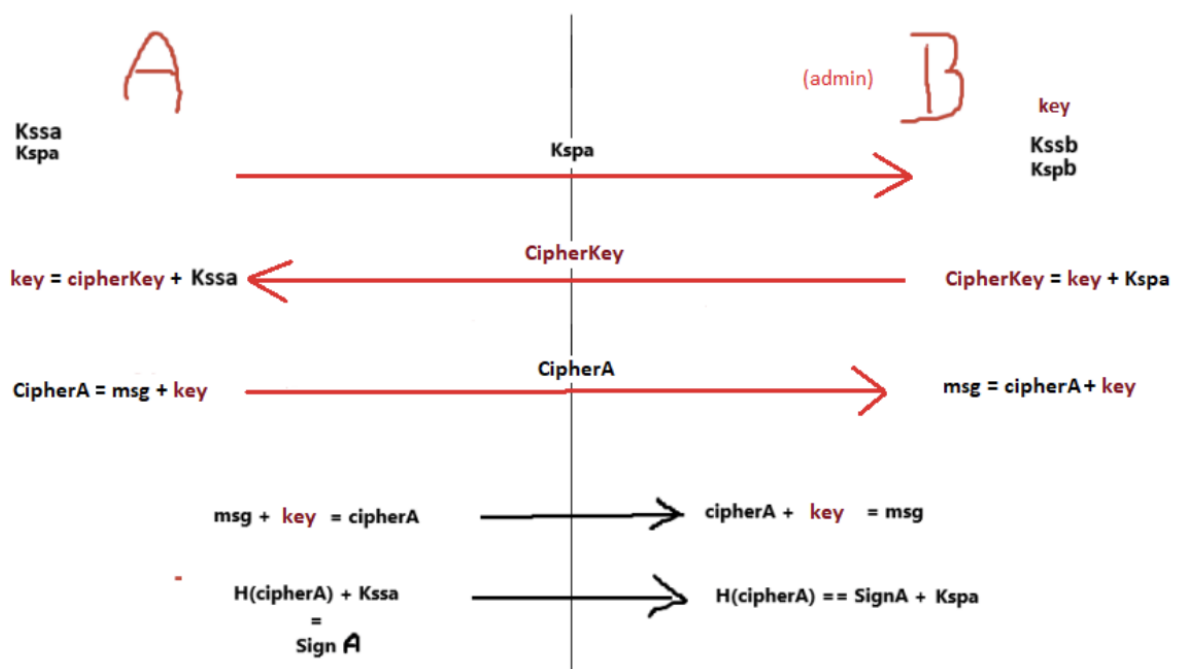


FIGURE 14 – Protocole de sécurité